# Modern Cloud Computing

## Lecture 22: Cloud and Edge I

Chongzhou Fang

Rochester Institute of Technology

# Today's Topics

- **MAUI** (MobiSys '10): energy-aware, method-level offload
- **COMET** (OSDI '12): transparent offload with distributed shared memory

# The Device-Cloud Gap

**Mobile / IoT devices:**

- Battery powered
- Slow CPU, little RAM
- Rich sensors: camera, mic, GPS
- Wireless, intermittent links

**Cloud servers:**

- Many fast cores, plentiful RAM
- No sensors, no user
- Stable, wired network

> Can a device *borrow* the cloud's resources when it needs them?

# Energy Problem

- MAUI's experiment: HTC Fuze, 1340 mAh battery
- Synthetic app: bulk Wi-Fi transfer + full CPU + backlight on
- Fully charged battery drained in **1 hour 20 minutes**

Demand keeps growing:

- Video games, streaming video
- Continuous sensing: navigation, pedometers, location-based apps

Battery capacity is *not* keeping up with the demand.

(Source: Cuervo et al., MAUI, MobiSys 2010, Sec. 2.1)

# Computation Offloading

**Idea:** run the expensive part of a program on a remote server, and ship back the result.

- Already in specialized products, e.g. speech and translation services

Two questions every offloading system must answer:

1. **What** to offload, and **when**?
2. **How** to move code and state, and keep the program correct?

# Two Classic Approaches

**Programmer-partitioned**

- Developer decides what runs remotely
- Fine-grained: only offload what pays off
- Large savings
- High programmer burden

**Process / VM migration**

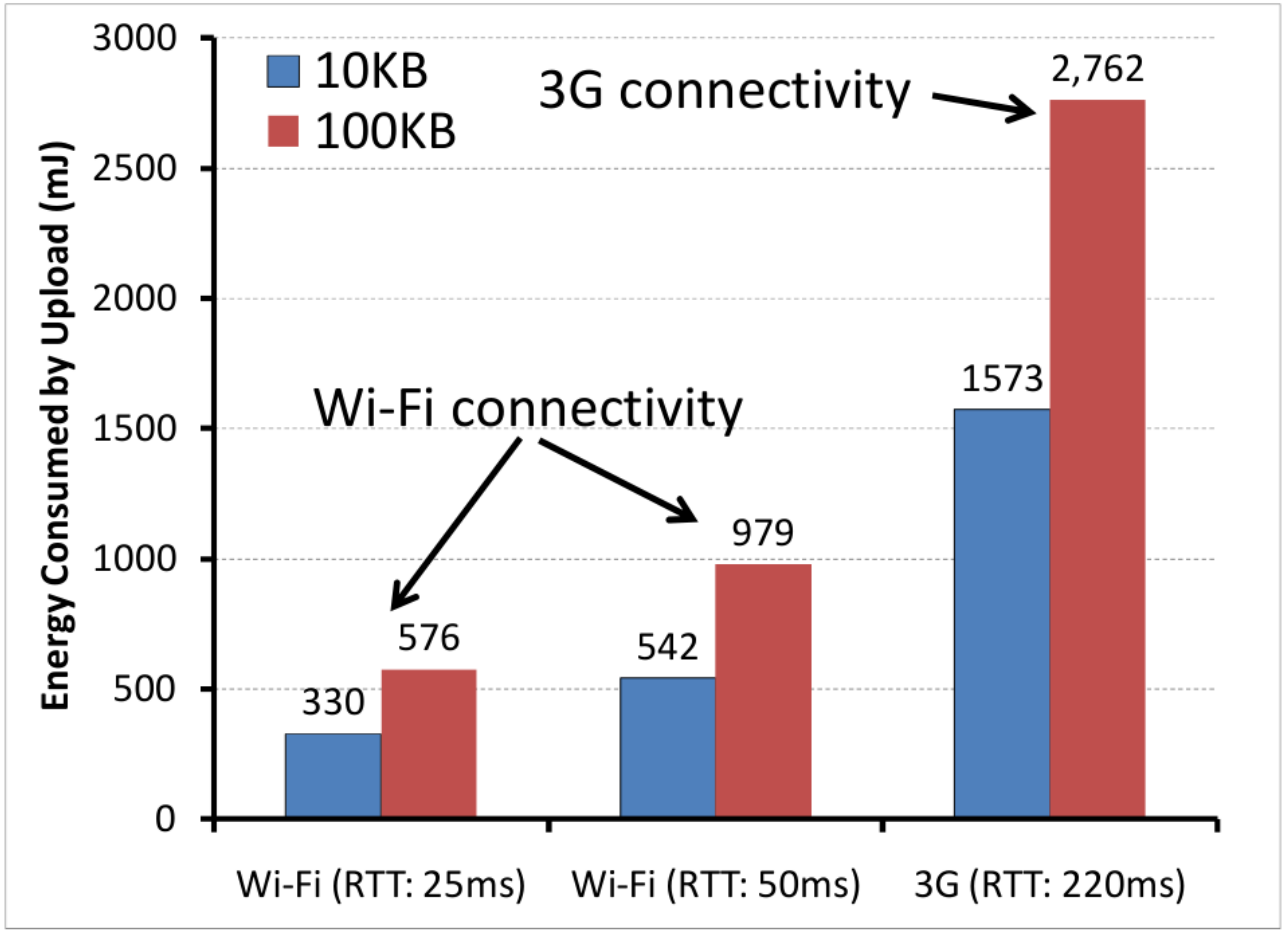
- Move the whole app or OS
- No app changes needed
- Coarse-grained: all state is shipped
- Smaller savings

> MAUI and COMET both try to get fine-grained offload with little or no programmer effort.

# Energy Cost

- CPU energy saved must exceed radio energy spent
- Uploads from an HTC Fuze: 3G uses **3x** the energy of Wi-Fi at 50 ms, **5x** at 25 ms

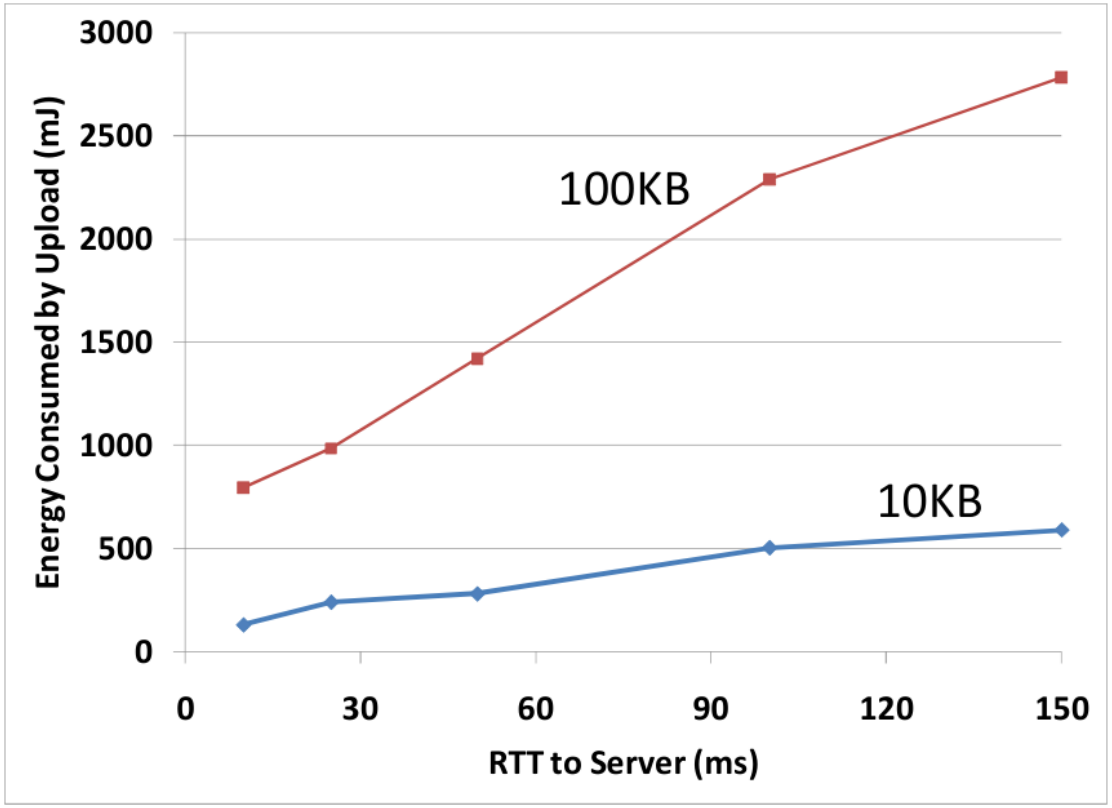
(Source: MAUI, MobiSys 2010, Fig. 1)



# Latency Cost

- Energy grows almost linearly with RTT
- 10 KB: energy almost doubles from 10 ms to 25 ms RTT
- Doubles again by 100 ms

(Source: MAUI, MobiSys 2010, Fig. 2)



# Part 1: MAUI

## Making Smartphones Last Longer with Code Offload

Cuervo, Balasubramanian, Cho, Wolman, Saroiu, Chandra, Bahl

MobiSys 2010

# MAUI: Goals

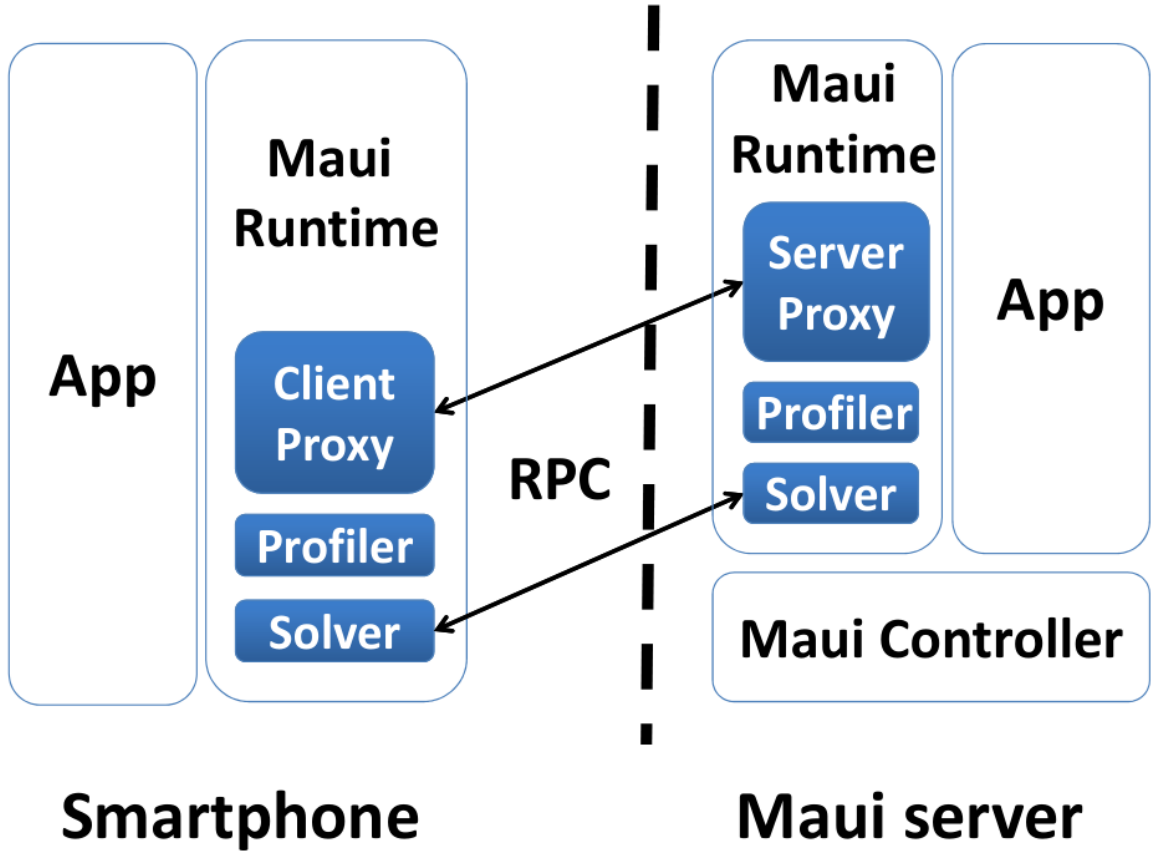
- **Primary goal:** save energy on the smartphone
- Fine-grained offload at the **method** level
- Minimal programmer burden
- Decide **at runtime**, per method invocation
- Adapt to changing network conditions

Implementation: a **managed runtime** (Microsoft .NET CLR)

# MAUI Architecture

- **Proxy:** control and state transfer
- **Profiler:** measures program and network
- **Solver:** making offloading decisions (runs on the server to save phone energy)
- **Controller:** authentication, resource allocation

(Source: MAUI, MobiSys 2010, Fig. 3)



# Partitioning: the `[Remoteable]` Annotation

Developer marks methods that are *safe* to run remotely:

```
public interface IEnemy {
  [Remoteable] bool SelectEnemy(int x, int y);
  [Remoteable] void ShowHistory();
  void UpdateGUI();
}
```

Do **not** mark as remoteable:

- User interface code
- Device I/O that only makes sense locally (GPS, accelerometer)
- Code with external side effects that break on re-execution (e.g. a purchase)

# Annotation Design Decisions

Offloading is optional

- Developer ensures **correctness**
- MAUI's solver optimizes **energy / performance** at runtime

Why opt-in (mark remoteable) instead of opt-out (mark local)?

- Forgetting a `[Remoteable]` only costs performance
- Forgetting a "local" tag could break the program

> A missed annotation should not affect correctness.

# Moving State

MAUI generates a wrapper for each remoteable method:

```
public interface IEnemyService {
  MAUIMessage<AppState, bool> SelectEnemy(AppState state, int x, int y);
  MAUIMessage<AppState, MauiVoid> ShowHistory(AppState state);
}
```

- One extra input (state to server), one extra return (state back)
- Conservative: all member variables of the object, plus static state
- **Incremental deltas**: send only what changed since the last offload
- Limitation: methods from only **one thread** offloaded at a time

# Handling Failures

- Detect with a simple **timeout**
- On disconnect, control returns to the local proxy, which either:
  - re-invokes the method locally, or
  - retries on another MAUI server

Cost of a failure: the energy spent shipping state to the server

# The MAUI Profiler

Offload decisions depend on three things:

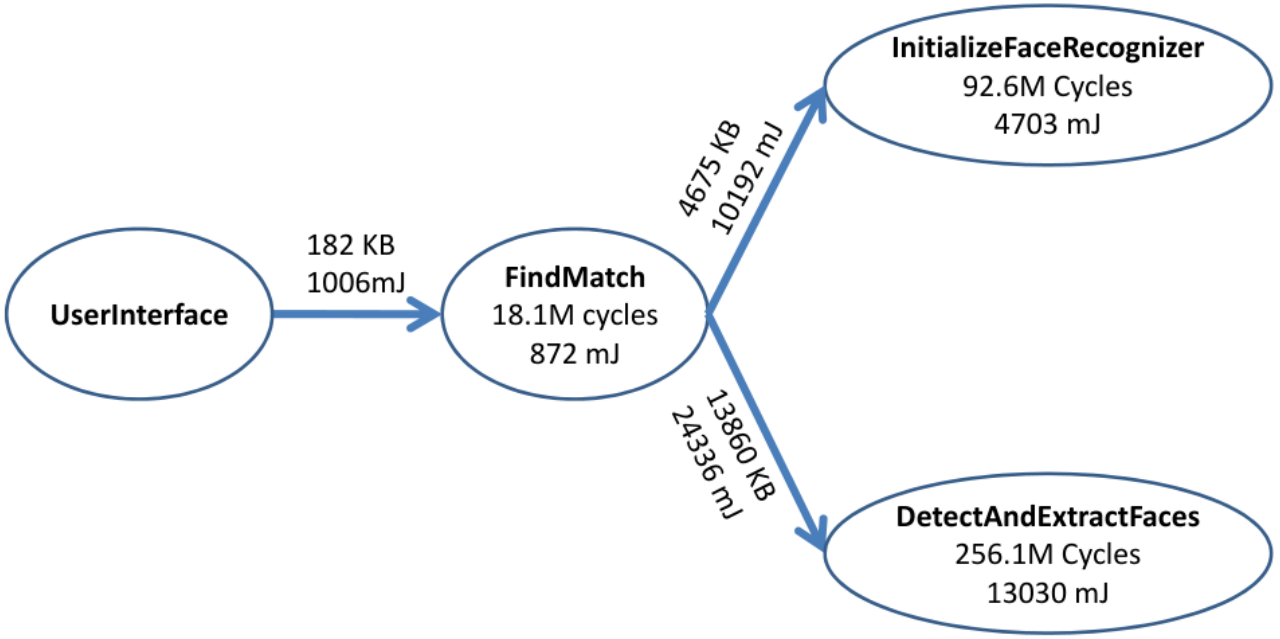
| Profile | What | When |
|---|---|---|
| **Device** | Energy model of CPU and radios | Once, with a hardware power meter |
| **Program** | State size, run time, CPU cycles per method | Continuously |
| **Network** | Bandwidth, latency | Continuously |

- CPU energy: linear model in CPU cycles, median error under 6%
- Past invocations of a method predict future ones
- Network: time a 10 KB TCP transfer; captures bandwidth and latency together

# The MAUI Solver

- Vertex: method, with CPU and energy cost
- Edge: state size and transfer energy

(Source: MAUI, MobiSys 2010, Fig. 8)



# # The MAUI Solver (Cont'd)

$I_v = 1$ if method $v$ runs remotely, $0$ if locally.

$$\max \sum_{v \in V} I_v E_v^l \; - \sum_{(u,v) \in E} |I_u - I_v| \, C_{u,v}$$

$$\text{s.t.} \sum_{v \in V} \left( (1 - I_v) T_v^l + I_v T_v^r \right) + \sum_{(u,v) \in E} |I_u - I_v| \, B_{u,v} \leq L, \qquad I_v \leq r_v$$

- Objective: energy saved minus energy spent on state transfer
- $L$: latency bound, default 5% above all-local execution
- $r_v$: only remoteable methods may be offloaded
- Re-solved periodically

# Part 2: COMET

## Code Offload by Migrating Execution Transparently

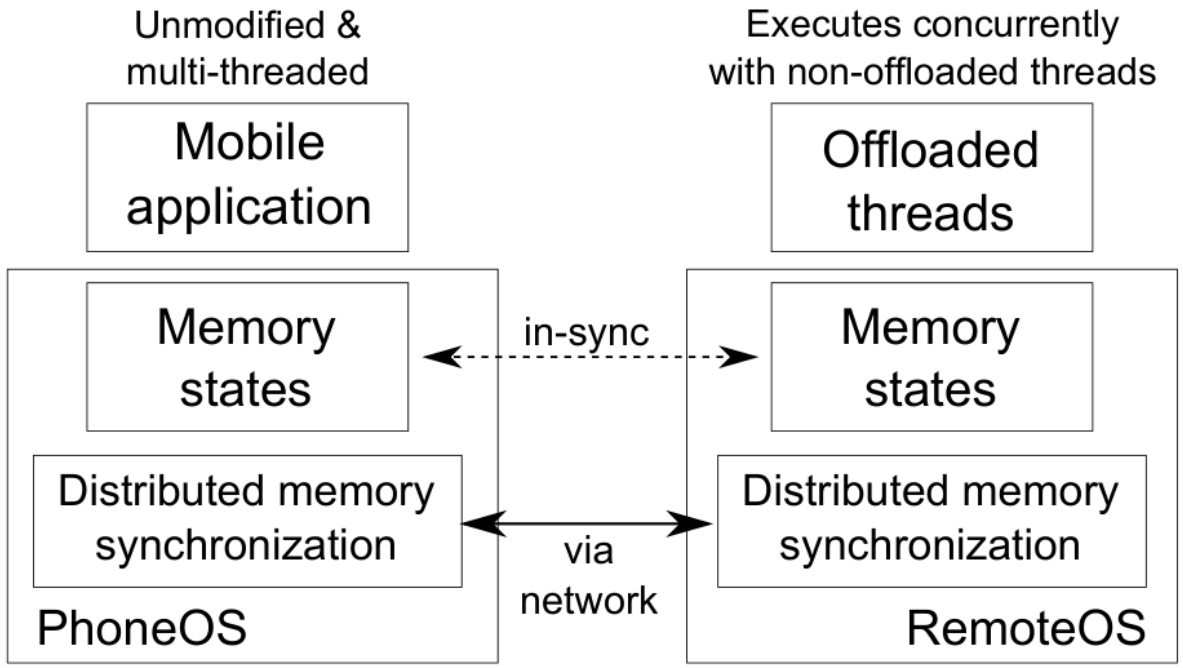
Gordon, Jamshidi, Mahlke, Mao, Chen

OSDI 2012

# COMET: Goals

- Require only the program binary
- Run multi-threaded programs correctly
- Improve computation speed
- Resist network and server failures
- Generalize to existing apps

Focus: **how** to offload




(Source: Gordon et al., COMET, OSDI 2012, Fig. 1)

# Key Idea: Distributed Shared Memory

**DSM:** several machines share one logical memory.

- Classic DSM (late 1980s on): clusters of workstations, scientific computing
- COMET: one Dalvik VM spanning a phone and a server
- Heap, stacks, and lock state kept consistent
- Threads migrate freely between the two endpoints

What is different for phones:

- High-latency, low-bandwidth wireless link
- Interactive apps with tight response requirements

# The Java Memory Model

Defines which writes a read must observe, via **happens-before**:

- Within a thread: total order
- Across threads: established by synchronization
  - Acquiring a lock last released by another thread
  - Volatile accesses, starting a thread

**Implication for COMET:**

- Endpoints only need to synchronize when a happens-before edge crosses machines
- Between those points, both sides can run with *no* communication

# Field-Level DSM

Each side keeps its own copy of the heap and marks every field it writes as **dirty**.

Example: one `Player` object, changed on both sides before a sync

| Field | Phone copy | Server copy | After syncing |
|---|---|---|---|
| `x` | **7** (dirty) | 3 | 7 |
| `score` | 10 | **25** (dirty) | 25 |
| `hp` | **90** (dirty) | **80** (dirty) | 90 or 80 |

- A sync sends **only the dirty fields**
- Data race: Java allows either value to win
- Payoff: both sides run freely, with no messages until a sync is needed

# VM-Synchronization

Directed operation: **pusher** to **puller**

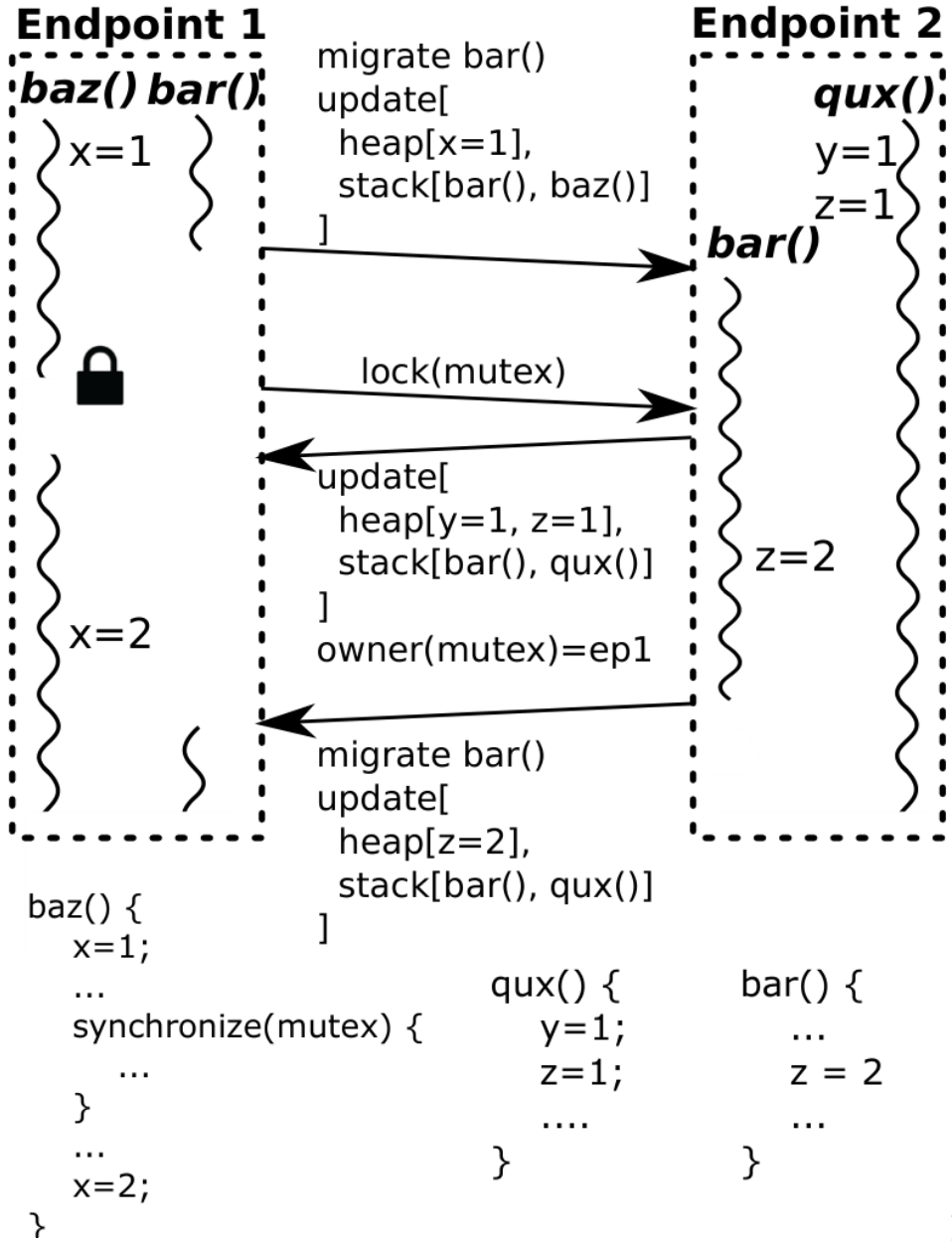
- Heap: all dirty fields
- Stacks: frames, PCs, registers
- Bytecode, class init state

Used for:

- Thread migration
- Lock ownership transfer

Only changes since the last sync are sent

(Source: COMET, OSDI 2012, Fig. 2)



# τ-Scheduling: When to Migrate

- Track how long a thread runs on the client without calling client-only (native) code
- Migrate once that time exceeds **τ**
- τ starts at 2 x RTT, then 2 x average VM-sync time

**Worst-case argument:**

- The thread already made τ of progress locally
- A wrong migration costs about τ to come back
- So mistakes cost at most about 2x the local run time